

Sumário

Atividade 03 — Protótipo individual com ESP32 e Wokwi	1
1. Identificação	1
2. Responsabilidade individual negociada	2
3. Relação com as Atividades 01 e 02	2
3.1 O cenário do grupo	2
3.2 De onde vem este recorte	3
3.3 Rastreabilidade do contrato de evento	3
3.4 Evoluções declaradas em relação às atividades anteriores	3
3.5 O que permanece coerente: privacidade por construção	3
3.6 O que ficou de fora, deliberadamente	4
4. Fenômeno, entrada, unidade e faixa	4
4.1 Declaração de fidelidade (seção 4 da atividade)	4
4.2 Grandezas representadas	4
5. Circuito e componentes	4
6. Contrato do evento	5
6.1 Tipos de evento emitidos	5
6.2 Campos de diagnóstico do perfil de sensoramento	5
6.3 Limitação declarada do tempo	6
7. Estado, regra e atuação	6
7.1 Máquina de estados	6
7.2 Por que a decisão não é uma comparação instantânea	6
7.3 Atuação e comportamento seguro	6
8. Resultados dos testes	7
Teste 1 — Estado normal (o sistema não atua)	7
Teste 2 — Decisão e atuação	8
Teste 3 — ADVERSARIAL (final de matrícula 4)	8
Testes complementares	10
Como reproduzir os testes automatizados	10
Compilação para o alvo ESP32	11
9. Limitações da simulação	11
10. Checklist de conformidade (seção 12 da atividade)	12
11. Declaração de uso de inteligência artificial generativa	12
Ferramentas utilizadas	12
Para quais partes	12
Como foi na prática	12
Uma adaptação que precisei fazer	13
O que aceitei das sugestões	13
O que recusei	13
O que precisei corrigir	13
O que ficou de saldo	13
12. Estrutura da entrega	14

Atividade 03 — Protótipo individual com ESP32 e Wokwi

Software para Sistemas Ubíquos — Instituto de Informática — UFG Prof. Dr. Otávio Calaça Xavier

1. Identificação

Campo	Valor
Estudante	João Victor Braga Queiroz
Matrícula	201810114 — final 4

Campo	Valor
Projeto do grupo	Balanceamento de fluxo de fila em entrada única de eventos, com validação de ingresso pela câmera do celular
Integrantes do grupo	Caio Castro Miranda · Jaime da Cruz Silva Junior · João Pedro de Brito Tomé · João Victor Braga Queiroz
Responsabilidade individual assumida	Sensoriamento e qualidade
Link do projeto no Wokwi	https://wokwi.com/projects/474800240502701057

2. Responsabilidade individual negociada

Assumi o perfil **Sensoriamento e qualidade**, cujo foco é “*leitura, faixa válida, calibração, filtragem, debounce, qualidade do sinal e detecção de falha*”.

Onde essa escolha aparece concretamente no que entreguei:

Aspecto do perfil	Onde está	Parâmetro
Leitura	amostragem periódica não-bloqueante do ADC	100 Hz (INTERVALO_AMOSTRAGEM_MS = 10)
Faixa válida	rejeição de leitura implausível antes de qualquer uso	ADC_MIN_VALIDO = 40, ADC_MAX_VALIDO = 4080
Calibração	conversão ADC→percentual com resolução declarada	12 bits, atenuação 11 dB, 0–4095 → 0–100 %
Filtragem	média móvel circular sobre o ADC	8 amostras (janela de 80 ms)
Debounce	botão de presença e chave de enlace	50 ms cada
Qualidade do sinal	limiar estrito + histerese + persistência	> 85 %, banda morta de 2 %, 1500 ms contínuos
Deteção de falha	leitura fora de faixa, enlace caído e dado vencido	TTL de 800 ms na amostra

A diferença verificável em relação aos colegas está em **onde o esforço foi gasto**: o meu protótipo dedica um estado inteiro da máquina (DADO_EXPIRADO), um atuador dedicado (LED amarelo), uma entrada dedicada (chave de enlace) e quatro campos do contrato de evento (dataAgeMs, dataValid, linkUp, expirations) exclusivamente à **procedência e à validade do dado** — não à regra de negócio da catraca.

3. Relação com as Atividades 01 e 02

3.1 O cenário do grupo

O grupo modelou o **balanceamento de fluxo de fila em entrada única de eventos** (shows e jogos geridos por plataforma de ticketeira). Na Atividade 01, câmeras top-down contam pessoas em cada fila e um serviço central calcula a vazão móvel para recomendar, num painel de LED, a fila menos congestionada. Na Atividade 02 o cenário evoluiu: a validação de ingresso passou a ser feita pela **câmera do celular/tablet** no ponto de validação, em modo QR ou facial, publicando os eventos ValidacaoRealizada e DispositivoHeartbeat via MQTT para um serviço de névoa que aplica janela deslizante de 120 s e histerese.

3.2 De onde vem este recorte

A Atividade 02 define que o celular decide localmente e publica apenas resultado: `validado | nao_validado` — **um booleano**. Mas alguém precisa produzir esse booleano, e a qualidade da captura é justamente o que determina se ele pode ser confiável.

Este protótipo implementa a **camada de sensoriamento que antecede esse booleano**: como o controlador do ponto de validação avalia se a captura facial tem qualidade suficiente, por quanto tempo essa avaliação precisa se sustentar, e — o ponto central — como ele evita decidir com base numa medição que já não descreve mais a cena.

É o recorte natural para quem assumiu **Sensoriamento e qualidade**: os demais perfis (agregação na névoa, contrato de comunicação, painel de recomendação) ficam com os outros integrantes, e nenhum deles trata da procedência do dado no instante da decisão.

3.3 Rastreabilidade do contrato de evento

Os nomes de evento e os campos foram deliberadamente herdados da Atividade 02, para que este protótipo se encaixe no que o grupo já especificou:

Atividade 02 (contrato do grupo)	Atividade 03 (contrato mínimo exigido)
<code>ValidacaoRealizada</code>	<code>eventType: "ValidacaoRealizada"</code>
<code>DispositivoHeartbeat</code>	<code>eventType: "DispositivoHeartbeat"</code>
<code>fila_id</code>	<code>entityId</code>
<code>seq</code>	<code>sequence</code>
<code>modo (qr facial)</code>	<code>modo: "facial" (campo adicional preservado)</code>
<code>resultado (validado nao_validado)</code>	<code>state (VALIDADO REJEITADO DADO_EXPIRADO)</code>

Os eventos `sensor.dado_expirado`, `sensor.leitura_invalida` e `credencial.expirada` são **novos**, introduzidos por este protótipo como contribuição do perfil de sensoriamento ao modelo do grupo.

3.4 Evoluções declaradas em relação às atividades anteriores

As Atividades 01 e 02 são a base do projeto, não um contrato imutável. Duas decisões deste protótipo divergem do que estava escrito, e registro ambas explicitamente em vez de deixá-las implícitas:

a) Atuação sobre a catraca. A Atividade 01 (item 8) afirmava que o sistema “*apenas sinaliza uma recomendação — não atua diretamente sobre o mundo físico (não controla catracas, portas ou qualquer mecanismo automático)*”. Isso era verdade quando o único sensoriamento era a contagem de pessoas. A partir da Atividade 02, com a validação de ingresso acontecendo no ponto de entrada, passou a existir uma **decisão de acesso** — e uma decisão de acesso só é útil se tiver consequência. Este protótipo assume essa evolução: o ponto de validação sinaliza a liberação da passagem.

b) Score de qualidade da captura. Não aparece nas Atividades 01 e 02, que tratam a validação como binária. Este protótipo o introduz como o **insumo** que produz esse binário, tornando explícito um passo que estava implícito no modelo.

3.5 O que permanece coerente: privacidade por construção

A decisão de privacidade das Atividades 01 e 02 — *dado biométrico é processado e descartado localmente; só o resultado trafega* — é preservada integralmente. O que atravessa a fronteira do dispositivo neste protótipo é um **número de 0 a 100 e um estado**, nunca uma imagem. Nenhum campo do contrato permite associar o evento a uma pessoa específica.

3.6 O que ficou de fora, deliberadamente

Reconhecimento facial real, câmera, serviço de névoa, MQTT, janela deslizante de 120 s e painel de recomendação. A seção 2 da Atividade 03 classifica esses itens como evolução futura e recomenda “*um recorte pequeno que funciona, possui estado e pode ser testado*” em vez de uma arquitetura extensa apenas descrita.

Num projeto paralelo cheguei a montar uma simulação **fullstack** da operação por curiosidade, um backend recebendo os eventos do dispositivo e um painel web mostrando o estado das filas em tempo real. Ficou funcionando, mas **decidi não enviar**. A atividade pede um protótipo de borda com estado observável e testável, e anexar uma aplicação web inteira só faria o avaliador procurar a entrega no meio do que não foi pedido. Fica como caminho de integração para as próximas etapas do projeto do grupo.

4. Fenômeno, entrada, unidade e faixa

4.1 Declaração de fidelidade (seção 4 da atividade)

Não há aquisição nem validação de nenhuma grandeza física real neste protótipo. Não há câmera, não há reconhecimento facial e não há biometria.

O potenciômetro é uma **entrada substituta** que representa um valor **fictício** de “score de qualidade da captura facial”, e a chave deslizante representa, também de forma fictícia, o enlace de rede com o aplicativo do celular. Renomear a entrada no código não a transforma em sensor real.

4.2 Grandezas representadas

Entrada	Componente	Grandeza fictícia representada	Unidade	Faixa válida
GPIO 34	Potenciômetro	Score de qualidade da captura facial	%	ADC 40-4080 ($\approx$ 1 % a 99 %)
GPIO 4	Push button	Presença de pessoa na catraca	booleano	—
GPIO 5	Chave deslizante	Enlace de rede com o app do celular	booleano	—

Por que a faixa válida não é 0-4095: um aplicativo em funcionamento nunca reporta score exatamente zero nem saturação total. Leituras fora de [40, 4080] são tratadas como **leitura implausível** (falha de enlace ou de sensor) e **descartadas**, em vez de serem interpretadas como “score muito baixo”. Essa distinção importa: um score baixo é uma informação válida sobre uma captura ruim; uma leitura implausível é ausência de informação.

5. Circuito e componentes

Componente	Pino	Papel
ESP32 DevKit v1	—	unidade de processamento
Push button	GPIO 4 (INPUT_PULLUP)	presença na catraca
Potenciômetro	GPIO 34 (ADC1_CH6)	score fictício 0-100 %
Chave deslizante	GPIO 5	enlace com o app (ativo / caído)
LED verde + 220 Ω	GPIO 18	acesso liberado
LED vermelho + 220 Ω	GPIO 19	acesso negado

Componente	Pino	Papel
LED amarelo + 220 Ω	GPIO 21	dado expirado / falha de leitura

O GPIO 34 é *input-only* no ESP32 e não aceita `pinMode()`; é usado apenas como entrada analógica, com `analogReadResolution(12)` e atenuação de 11 dB para cobrir a faixa de 0 a 3,3 V.

6. Contrato do evento

Todo evento é impresso no monitor serial em JSON. Os oito primeiros campos são o contrato mínimo exigido pela seção 5.2 da atividade.

```
{
  "eventType": "ValidacaoRealizada",
  "deviceId": "esp32-catraca-03",
  "entityId": "portao-2-catraca-3",
  "eventTimeMs": 1851,
  "sequence": 3,
  "value": 91,
  "unit": "%",
  "state": "DADO_EXPIRADO",
  "reason": "dado_expirado",
  "dataAgeMs": 801,
  "dataValid": false,
  "linkUp": false,
  "windowMs": 0,
  "expirations": 1,
  "dropouts": 0
}
```

6.1 Tipos de evento emitidos

eventType	Quando
ValidacaoRealizada · state: VALIDADO	janela contínua de 1500 ms cumprida com dado válido
ValidacaoRealizada · state: REJEITADO	presença perdida ou qualidade insuficiente
ValidacaoRealizada · state: DADO_EXPIRADO	tentativa de passagem com o dado fora da validade
sensor.dado_expirado	o score em memória ultrapassou o TTL de 800 ms
sensor.leitura_invalida	ADC fora da faixa plausível
credencial.expirada	a autorização concedida ultrapassou o TTL de 3000 ms
DispositivoHeartbeat	a cada 5 s, para tornar visível o silêncio do dispositivo

6.2 Campos de diagnóstico do perfil de sensoriamento

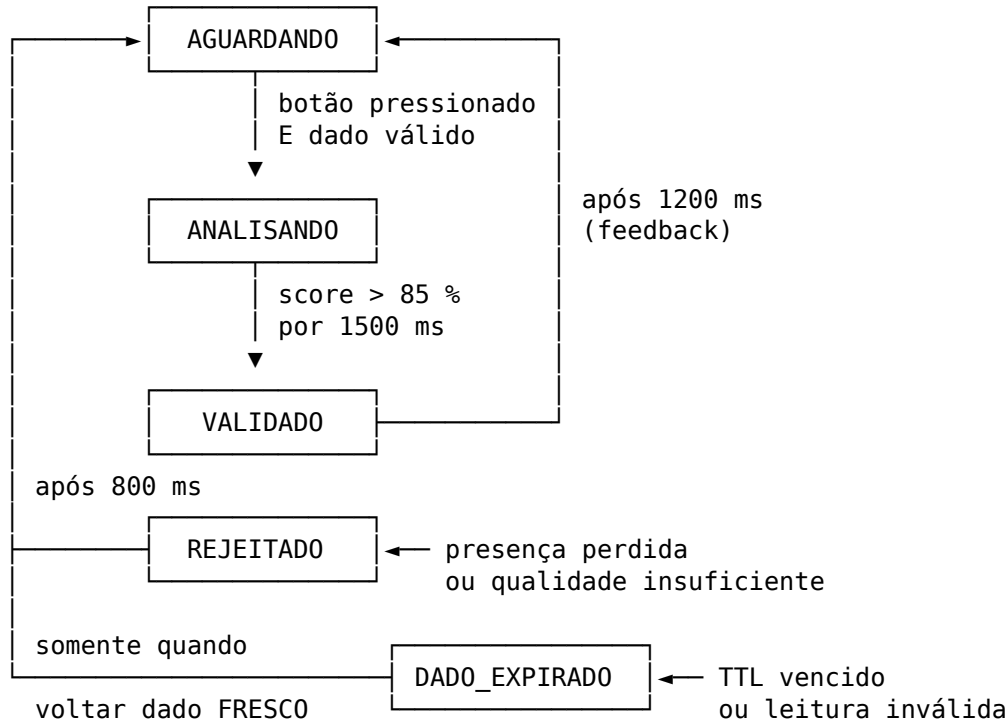
`dataAgeMs` (idade do dado no instante da decisão), `dataValid` (o dado estava dentro do prazo?), `linkUp` (o app estava enviando?), `windowMs` (quanto da janela contínua havia decorrido), `expirations` e `dropouts` (contadores acumulados). São esses campos que tornam a decisão **auditaável a posteriori**: lendo só o JSON dá para saber se a catraca abriu com dado fresco ou não.

6.3 Limitação declarada do tempo

eventTimeMs é millis(), o tempo decorrido desde o boot do ESP32 — **não é relógio de parede sincronizado**. Consequências: eventos de dispositivos diferentes não são diretamente comparáveis no tempo; um reboot reinicia a contagem; e há overflow de uint32_t após aproximadamente 49 dias de operação contínua. Todas as diferenças de tempo no firmware são calculadas como agora - marco, forma que permanece correta através do overflow.

7. Estado, regra e atuação

7.1 Máquina de estados



7.2 Por que a decisão não é uma comparação instantânea

A atividade proíbe que a decisão seja apenas valor > limite (seção 5.3). A regra implementada acumula **cinco** características de estado e qualidade — todas precisam ser satisfeitas simultaneamente:

1. **Persistência** — o score precisa se manter acima do limiar por 1500 ms **contínuos**. Intervalos picotados não são somados: qualquer interrupção zera o marco tsInicioJanela.
2. **Combinação de duas entradas** — score do potenciômetro **e** presença do botão, simultaneamente.
3. **Debounce** — 50 ms no botão e na chave, contra repique mecânico.
4. **Histerese** — entra acima de 85 %, só sai abaixo de 83 %, evitando oscilação de sinalização na borda do limiar.
5. **Validade do dado** — o score precisa ter sido medido há menos de 800 ms. Esta é a característica exercitada pelo teste adversarial.

O limiar é comparado de forma **estrita** (> 85, não >=). Um score estabilizado em exatamente 85 % nunca valida — comprovado no Teste 5.

7.3 Atuação e comportamento seguro

Estado	LED	Semântica
AGUARDANDO	todos apagados	ocioso
ANALISANDO	verde piscando (contando) / vermelho (qualidade baixa)	analizando
VALIDADO	verde fixo	passagem liberada
REJEITADO	vermelho fixo	passagem negada
DADO_EXPIRADO	amarelo piscando	<i>o sistema não sabe o estado atual da cena</i>

Comportamento seguro: quando a entrada é inválida ou insuficiente, a decisão é sempre **negar**. O amarelo é deliberadamente distinto do vermelho: vermelho significa “avaliei e re-provei”; amarelo significa “não tenho informação confiável para avaliar”. Confundir os dois é justamente o erro que o teste adversarial expõe.

Prevenção de oscilação e de repetição: a histerese impede tremulação na borda do limiar; `falhaJaReportada` impede que a mesma falha de leitura seja publicada a cada ciclo de 10 ms; e a saída de `DADO_EXPIRADO` exige um dado fresco de verdade — o sistema **não se recupera apenas pela passagem do tempo**.

8. Resultados dos testes

Os três testes obrigatórios da seção 7 foram executados. Para cada um estão registrados estado inicial, entradas aplicadas, tempos, resultado esperado, resultado observado e a explicação da decisão.

De onde vêm os números. Cada teste foi executado em duas frentes independentes:

- **No Wokwi**, manualmente, girando o potenciômetro e apertando o botão. São essas execuções que estão nos arquivos de evidências/.
- **Na bancada automatizada**, que compila e roda o mesmo `sketch.ino` sobre um relógio virtual de 1 ms, com as entradas fixadas por programa.

Os traços numéricos transcritos abaixo são os da **bancada**, porque ali os tempos são determinísticos e reproduzíveis: o botão é pressionado exatamente em $t=100$ ms e o ADC vale exatamente 3767. No Wokwi os valores diferem um pouco — o score parou em 93 % em vez de 91 %, porque posicionar um potenciômetro arrastando o mouse não acerta um valor exato — mas **o que está sendo testado coincide nas duas**: a janela de 1500 ms, o TTL de 801 ms e a recusa de liberar com dado vencido aparecem iguais nos dois lugares. Onde houver divergência de valor, ela está apontada no teste correspondente.

Teste 1 — Estado normal (o sistema não atua)

Estado inicial	AGUARDANDO, enlace ativo, ninguém na catraca
Entradas aplicadas	potenciômetro em ~60 %, botão nunca pressionado, 5 s de observação
Resultado esperado	nenhum LED de decisão aceso, nenhum evento de acesso
Resultado observado	permaneceu em AGUARDANDO; nenhum <code>ValidacaoRealizada</code> + <code>state</code> : VALIDADO; nenhuma autorização concedida; nenhum <code>sensor.dado_expirado</code> (o enlace ativo mantém o dado fresco)

Explicação da decisão: faltam simultaneamente a presença (botão) e a qualidade ($60\% < 85\%$). O firmware continua amostrando e renovando o carimbo de frescor, mas nenhuma transi-

ção é disparada. Isso demonstra que a saída não é consequência automática da alimentação do circuito.

Evidência no Wokwi: evidencias/teste-normal.png — mostra estado=AGUARDANDO score=60% valido=SIM enlace=ATIVO botao=SOLT0 com os três LEDs apagados.

Teste 2 — Decisão e atuação

Estado inicial	AGUARDANDO, enlace ativo, score estável em ~92 %
Entradas aplicadas	botão pressionado em t=100 ms e mantido por 2,5 s
Resultado esperado	LED verde aceso e um único ValidacaoRealizada + state: VALIDADO, nunca antes de 1500 ms de janela
Resultado observado	janela iniciada em t=151 ms; VALIDADO em t=1651 ms ; exatamente 1 evento ValidacaoRealizada + state: VALIDADO; value = 91 %; sequence monotônico

Saída do monitor serial:

```
[ESTADO] AGUARDANDO -> ANALISANDO
[JANELA] Score 91% acima do limiar – contando 1500 ms continuos
[ESTADO] ANALISANDO -> VALIDADO
{"eventType":"ValidacaoRealizada","deviceId":"esp32-catraca-03",
 "entityId":"portao-2-catraca-3","eventTimeMs":1651,"sequence":1,
 "value":91,"unit":"%","state":"VALIDADO",
 "reason":"janela_continua_atingida","dataAgeMs":1,"dataValid":true,
 "linkUp":true,"windowMs":1500,"expirations":0,"dropouts":0}
```

Explicação da decisão: 1651 – 151 = **1500 ms exatos** de janela contínua. O atraso de 51 ms entre o toque no botão e o início da janela é o debounce. O score reportado é 91 %, e não 92 %, porque a conversão ADC→percentual é inteira e trunca: $\lfloor 3767 \times 100 / 4095 \rfloor = 91$.

Evidência no Wokwi: evidencias/teste-decisao.png — LED verde fixo e o JSON ValidacaoRealizada com "state":"VALIDADO" e "windowMs":1500. Ali o score ficou em **93 %** e o eventTimeMs é grande porque a simulação já rodava havia minutos quando o teste foi feito; o que importa comparar é o windowMs, que vale 1500 nos dois casos.

Teste 3 — ADVERSARIAL (final de matrícula 4)

Situação obrigatória: *“Informação que permanece armazenada depois de perder a validade.”*

3.1 Por que uma implementação ingênua decidiria errado A forma direta de escrever este firmware é guardar o score numa variável global e consultá-la quando alguém encosta na catraca:

```
int ultimoScore = 0; // implementacao INGENUA
void loop() {
  ultimoScore = map(analogRead(34), 0, 4095, 0, 100);
  if (digitalRead(4) == LOW && ultimoScore > 85) abrirCatraca();
}
```

O problema não aparece no caminho feliz. Ele aparece quando **a fonte do dado para de responder**: o usuário bloqueia a tela, cobre a câmera, o Wi-Fi cai, o app é morto pelo sistema operacional. ultimoScore continua valendo 92 indefinidamente, porque **nada no código o**

invalida — e a catraca abre com base numa medição que descreve uma cena que já não existe mais. Pior: o valor de 92 % é *plausível*, então nenhuma verificação de faixa o pega. É informação que sobreviveu à própria validade.

3.2 Qual mecanismo foi implementado Carimbo de tempo por amostra, com TTL, mais estado observável.

1. Toda amostra aceita registra `tsAmostraValida = millis()`.
2. `dadoFresco()` compara `millis() - tsAmostraValida` com `VALIDADE_AMOSTRA_MS = 800`.
3. **Nenhuma decisão consulta o score sem antes consultar a idade dele.**
4. Vencido o prazo, o sistema entra em `DADO_EXPIRADO`: LED amarelo piscando, evento JSON com `dataValid: false`, e decisão segura = negar.
5. A saída desse estado exige **dado fresco de verdade** — não basta esperar.
6. A **autorização concedida** também tem prazo (`VALIDADE_AUTORIZACAO_MS = 3000`), impedindo que uma validação antiga continue valendo.

Ponto central: o score **continua na memória** (é observável na telemetria). O que muda é que ele deixa de ser **utilizável**. O firmware não apaga a informação; ele passa a saber de quando ela é.

3.3 Saídas observadas 3a — o enlace cai e o dado envelhece:

Estado inicial	AGUARDANDO, score 91 %, enlace ativo
Entradas aplicadas	chave de enlace desligada em <code>t=1000 ms</code> ;
Resultado esperado	potenciômetro não é tocado o score permanece em memória, mas é marcado inválido ao passar de 800 ms
Resultado observado	score preservado em 91 %; <code>sensor.dado_expirado</code> com <code>dataAgeMs = 801</code> , <code>dataValid: false</code> , <code>linkUp: false</code> ; estado <code>DADO_EXPIRADO</code>

```
[EXPIRA] Score de 91% perdeu a validade (idade 801 ms > 800 ms) – dado marcado como NAO CONFIA
[ESTADO] AGUARDANDO -> DADO_EXPIRADO
{"eventType":"sensor.dado_expirado", ... "value":91,"state":"DADO_EXPIRADO",
 "reason":"ttl_amostra_excedido","dataAgeMs":801,"dataValid":false,"linkUp":false, ...}
```

3b — alguém tenta passar usando o dado velho:

Entradas aplicadas	enlace caído em <code>t=1000 ms</code> ; botão pressionado em <code>t=2500 ms</code> e mantido
Resultado esperado	acesso negado, apesar de o score em memória ser 91 %
Resultado observado	nenhuma ValidacaoRealizada + state: VALIDADO ; permaneceu em <code>DADO_EXPIRADO</code> ; nenhuma autorização ativa

Esta é a evidência central: **a implementação ingênua abriria a catraca aqui**, porque `ultimoScore > 85` continua verdadeiro.

Evidências no Wokwi:

- `evidencias/teste-adversarial.png` — o momento da expiração: as linhas [SINAL] com idade= subindo de 234 ms para 734 ms, o [EXPIRA] disparando em **801 ms**, e o JSON `sensor.dado_expirado` com `"value":93` convivendo com `"dataValid":false`. O LED amarelo aceso.
- `evidencias/teste-adversarial-3b.png` — a tentativa de passagem: `botao=PRESSIONADO`, score de 93 % ainda em memória, e mesmo assim `janela=0/1500ms` e LED verde apagado. Como ali já se passaram quase 8 s desde a expiração, esse print também cobre o item 3d: o sistema não se recuperou sozinho com a passagem do tempo.

3c — a autorização também expira:

Entradas aplicadas	validação completa aos 1651 ms; usuário não passa
Resultado observado	credencial.expirada em t=4651 ms — exatamente 3000 ms após a concessão

3d — a recuperação exige dado fresco, não apenas tempo:

Entradas aplicadas	enlace cai em t=1000 ms, volta em t=3000 ms
Resultado observado	[REARME] em t=3050 ms (a chave leva 50 ms de debounce); antes disso o sistema permaneceu em DADO_EXPIRADO mesmo com o tempo passando

Testes complementares

#	Teste	Resultado
4	Faixa válida — ADC cai para 5 (fora de [40,4080])	sensor.leitura_invalida emitido; score preservado em 91 % em vez de ser contaminado para 0 %; estado observável de falha
5	Limiar exato — score estabilizado em exatamente 85 %	nenhuma validação em 6 s, confirmando a comparação estrita > 85
6	Queda transitória — score cai 150 ms sem soltar o botão	rejeição em t=1220 ms descartando 1069 ms acumulados; validação só em t=3522 ms. Se os intervalos fossem somados, teria validado por volta de 1650 ms
7	Ruído de ADC — ± 60 LSB de ruído sobre 92 %	validou normalmente, sem rejeição espúria: a média móvel de 8 amostras absorveu o ruído

O teste 5 merece destaque: acertar exatamente 85 % arrastando o potenciômetro com o mouse é praticamente impossível. Ele só é verificável na bancada automatizada, onde o valor do ADC é fixado em 3500 ($\lfloor 3500 \times 100 / 4095 \rfloor = 85$).

Como reproduzir os testes automatizados

```
./tests/nativo/rodar.sh
```

O que essa bancada é: ela compila o **firmware real** — o harness inclui textualmente edge-esp32/sketch.ino, o mesmo arquivo que vai para o Wokwi — e o executa sobre um relógio virtual de 1 ms, com um *shim* que reimplementa a API do Arduino (`millis()`, `digitalRead()`, `analogRead()`, `Serial`) no host. Não há reescrita nem transliteração: o código exercitado é o código entregue.

São 10 cenários e 43 asserções, cada cenário num processo separado para garantir estado global limpo. Além do monitor serial, as asserções inspecionam o nível elétrico dos pinos dos LEDs, verificando que a atuação corresponde à decisão.

O firmware compila com `g++ -std=c++17 -Wall` sem nenhum aviso.

Evidência: `evidencias/bancada-testes.png` — a execução completa, com as 43 asserções e a faixa final. É a única evidência desta entrega que o avaliador pode reexecutar na própria máquina, e a única fonte dos Testes 5, 6 e 7, que não têm como ser reproduzidos à mão no simulador.

O que essa bancada não substitui: ela valida a lógica temporal, não o hardware. O shim não reproduz o ADC real do ESP32, o comportamento elétrico dos pinos nem o escalonamento do FreeRTOS. A execução no Wokwi continua sendo a evidência oficial da entrega.

Compilação para o alvo ESP32

```
./tests/compile-esp32.sh
```

O harness nativo compila com `g++` para o host, o que valida a lógica mas não a API do ESP32. Esta verificação complementar compila o mesmo `sketch.ino` com o **core esp32:esp32 3.3.11** — o mesmo que o Wokwi utiliza — para a placa `esp32:esp32:esp32doit-devkit-v1`, correspondente à `wokwi-esp32-devkit-v1` do `diagram.json`.

Resultado, com `--warnings all`:

```
Sketch uses 285216 bytes (21%) of program storage space. Maximum is 1310720 bytes.
Global variables use 22324 bytes (6%) of dynamic memory, leaving 305356 bytes
for local variables. Maximum is 327680 bytes.
```

Nenhum aviso emitido. O consumo de 21% de flash e 6% de RAM confirma que o protótipo cabe folgadoamente no alvo, e que o `snprintf` de 480 bytes usado para montar o payload não pressiona a memória.

9. Limitações da simulação

O que foi efetivamente validado, somando a execução no Wokwi e a bancada automatizada: a lógica temporal não-bloqueante, a máquina de estados, o debounce, o filtro de média móvel, a expiração por TTL e a coerência entre decisão e atuação. O Wokwi cobre a operação manual e a observação física dos LEDs; a bancada cobre os cenários que não dão para fazer à mão, como o limiar exato em 85 % e a queda transitória de 150 ms.

O que exigiria hardware e ambiente reais:

1. **Não há reconhecimento facial.** Nenhuma imagem é capturada ou processada. O “score de qualidade” é um valor arbitrário girado à mão num potenciômetro. Nada aqui sustenta qualquer afirmação sobre acurácia biométrica.
2. **Não há enlace de rede.** A chave deslizante simula a queda do enlace, mas não existe Wi-Fi, MQTT, HTTP ou app real. Perda de pacote, latência variável, retransmissão e reordenação — os problemas de verdade de um enlace — não são exercitados.
3. **O ADC do Wokwi é ideal.** Não reproduz a não-linearidade conhecida do ADC do ESP32 real, nem ruído térmico, nem deriva com a temperatura, nem a necessidade de calibração por unidade. Os valores de `ADC_MIN_VALIDO` e `ADC_MAX_VALIDO` teriam de ser recalibrados em hardware físico.
4. **O tempo do simulador não é o tempo real.** O Wokwi pode adiantar ou atrasar em relação ao relógio de parede conforme a carga da máquina do usuário. Os 1500 ms observados são tempo simulado.
5. **Não há relógio sincronizado.** `eventTimeMs` é `millis()` desde o boot, com overflow em ~49 dias, como detalhado em 6.3.
6. **O TTL de 800 ms é uma escolha de projeto, não uma medição.** Num sistema real ele deveria ser derivado da taxa de envio do app e da latência do enlace, com margem estatística — não escolhido de antemão.

7. **Segurança física não é tratada.** Carona (*tailgating*), apresentação de foto à câmera e coação estão fora do escopo.

10. Checklist de conformidade (seção 12 da atividade)

Item	Situação
Protótipo relacionado ao projeto do grupo	Sim — recorte do controlador da catraca
Responsabilidade individual declarada	Sim — Sensoriamento e qualidade (seção 2)
Implementação distinguível dos demais integrantes	Sim — estado, atuador, entrada e campos de evento dedicados à validade do dado
Circuito executa no Wokwi	Sim — testado em wokwi.com; monitor serial e três LEDs verificados
Pelo menos uma entrada e uma saída observável	Sim — 3 entradas, 3 LEDs
Monitor serial com evento JSON e sequência	Sim — campo sequence monotônico
Regra usa estado/tempo/qualidade, não comparação instantânea	Sim — cinco mecanismos (seção 7.2)
Três testes executados e documentados	Sim — seção 8
Teste adversarial corresponde ao final da matrícula	Sim — final 4 → validade expirada
Limitações da simulação explícitas	Sim — seção 9
Uso de IA declarado	Sim — seção 11
ZIP com código, circuito, relatório e evidências	Sim — sketch.ino, diagram.json, libraries.txt, relatorio.pdf e evidencias/
Link do Wokwi acessível	Sim — projeto salvo e público

11. Declaração de uso de inteligência artificial generativa

Ferramentas utilizadas

Gemini e Claude, este último pelo Claude Code no terminal.

Para quais partes

Usei o **Gemini no começo**, para organizar minha pesquisa sobre a atividade e montar um roteiro do que eu precisava fazer: quais eram as exigências, o que significava o teste adversarial do meu dígito, e em que ordem atacar cada parte. Foi ele que me deu o mapa.

Depois usei o **Claude Code** para escrever o firmware, montar o diagram.json, construir a bancada de testes e me ajudar na redação deste relatório — sempre com aquele roteiro como referência do que eu queria chegar, e tirando dúvidas pelo caminho.

Como foi na prática

Era a primeira vez que eu mexia com Wokwi e com simulação de microcontrolador, e isso pesou. Sofremos bastante para colocar a coisa para rodar: o monitor serial simplesmente não aparecia, e a causa só apareceu depois de muita tentativa — faltavam no diagram.json as duas conexões que ligam os pinos TX e RX à peça virtual do monitor. Nenhum template avisa isso; ou você já sabe, ou descobre comparando com um projeto que funciona.

A plataforma web deu tanto problema, e o prazo estava tão em cima, que instalei a **extensão do Wokwi no VS Code** para simular localmente. Foi uma saída de emergência, mas acabou ajudando: em algumas ocasiões foi rodando localmente que eu consegui isolar onde estava o erro, sem depender do site.

A maior parte do meu tempo não foi escrever código. Foi fazer os cenários de teste rodarem do jeito que o meu roteiro previa, e entender por que cada um falhava quando falhava. Foi aí que eu realmente aprendi como o protótipo funciona — não na hora em que o código apareceu, mas na hora em que ele não fazia o que eu esperava.

Uma adaptação que precisei fazer

A atividade pede um protótipo de um cenário especulativo, e o projeto do meu grupo não cabia direto numa placa simulada. O grupo trata a validação de ingresso como binária: validou ou não validou. Para ter o que sensoriar, tive que recuar um passo e prototipar o que **produz** esse binário — o score de qualidade da captura e o enlace com o app —, ambos como entradas substitutas.

Não é o sistema que o grupo descreveu. É o pedaço dele que dá para construir, observar e testar com um ESP32, um potenciômetro e três LEDs.

O que separa o meu protótipo do dos meus colegas é o **tema que eu assumi** — sensoriamento e qualidade — e a **forma como escolhi representar o reconhecimento facial**: não como um “validou ou não validou”, mas como um score contínuo que tem origem, tem prazo e pode envelhecer. Foi essa escolha que abriu espaço para tudo o que veio depois: o carimbo de tempo, o estado DADO_EXPIRADO, o LED amarelo e os campos de validade no evento.

O que aceitei das sugestões

Média móvel de 8 amostras e debounce de 50 ms para o ruído; separar DADO_EXPIRADO de REJEITADO em estados distintos; incluir dataAgeMs e dataValid no evento; tratar leitura fora da faixa como falha de sensor e não como score baixo.

O que recusei

Um LED bicolor para os dois estados de falha — “reprovado” e “não sei o que está acontecendo na frente da catraca” são situações diferentes, e quem opera precisa distinguir uma da outra olhando o painel. E a primeira bancada de testes, escrita em JavaScript, que descartei ao perceber que ela não testava o firmware que eu ia entregar.

O que precisei corrigir

Nem tudo que veio pronto estava certo, e três coisas eu só peguei porque fui conferir na fonte:

- **O teste adversarial estava trocado.** A primeira versão implementou “queda transitória de sinal”, que corresponde aos dígitos 0 a 3. Lendo o PDF da atividade, o meu dígito 4 é *“informação que permanece armazenada depois de perder a validade”* — outra coisa. O firmware teve de ser reescrito.
- **A bancada de testes não testava o código de verdade.** Era uma transliteração da máquina de estados para JavaScript, e por isso não reproduzia um bug de inicialização que existia no firmware real. Reescrevi a bancada para compilar e executar o sketch.ino com g++, e o defeito apareceu na primeira execução.
- **Uma depreciação de API que não existia.** Foi afirmado que a constante ADC_11db estaria depreciada nas versões novas do core ESP32. Abri o header do core instalado para conferir e não havia nada disso. Removi a afirmação.

O terceiro me marcou mais que os outros: era uma informação com forma de verdade — soava técnica, específica, plausível — e passaria por qualquer leitura que não abrisse o arquivo.

O que ficou de saldo

Confesso que foi divertido. Ver o LED amarelo começar a piscar exatamente 801 ms depois de eu desligar a chave, e o botão simplesmente não abrir a catraca com um score de 93 % ainda na memória, é um retorno bem mais concreto do que ler sobre validade de dado num slide. O Wokwi me despertou curiosidade: pretendo continuar mexendo nele, com sensores de verdade e outras placas.

12. Estrutura da entrega

atividade-03-joao-victor-braga-queiroz-201810114.zip

— sketch.ino	firmware (não-bloqueante, sem delay())
— diagram.json	circuito Wokwi
— libraries.txt	nenhuma biblioteca externa
— relatorio.pdf	este documento
— evidencias/	
— teste-normal.png	score 60 %, AGUARDANDO, nenhum LED aceso
— teste-decisao.png	LED verde + JSON VALIDADO com windowMs 1500
— teste-adversarial.png	[EXPIRA] em 801 ms, LED amarelo, dataValid false
— teste-adversarial-3b.png	botão pressionado com dado vencido: não libera
— bancada-testes.png	10 cenários, 43 asserções sobre o firmware real